

Build It Yourself

Step-by-step implementation guide

Spring Boot + MySQL

React + square red-and-white UI

Windows NFC Reader Agent

Sui Move + Testnet Worker

Deployment, testing and recovery

Build one verified milestone at a time.

Purpose, file location, code examples and acceptance checks.

Not certified for live hospital use; integration gates are explicit.

Contents

Click a chapter title to jump to its actual PDF page.

00 How to use this handbook	5
01 Roadmap and milestones	6
02 Architecture and explicit decisions	7
03 Install the Windows toolchain	8
04 Establish the project structure	9
05 Initialize Spring Boot	10
06 Local MySQL and persistence	11
07 Configure Backend and the first endpoint	12
08 Data model and migration order	13
09 Constraints and application layering	14
10 Define API contracts before pages	15
11 Authentication and roles	16
12 Session, CSRF and browser requests	17
13 Implement four registration categories	18
14 Hospital API and MRN validation	19
15 Queue, verification and rejection	20

Contents

Click a chapter title to jump to its actual PDF page.

16 Inventory and the issue transaction	21
17 Idempotency, return and overdue	22
18 Audit, reporting and administration	23
19 Initialize React, Vite and shadcn/ui	24
20 Frontend structure and request layer	25
21 Red-and-white square UI system	26
22 Mobile registration from QR to reference	27
23 Staff/admin UI and scan state	28
24 Hardware preparation and compatibility	29
25 Windows drivers and independent reading	30
26 Card provisioning and category parsing	31
27 Reader Agent and cloud scan jobs	32
28 Package and install the agent	33
29 What Sui proves and what it does not	34
30 Sui CLI and Testnet wallets	35
31 Design the Move package	36

Contents

Click a chapter title to jump to its actual PDF page.

32 Test and publish Move	37
33 TypeScript worker and outbox	38
34 Confirmation, retry and proof checking	39
35 Deployment topology and configuration	40
36 Railway Backend, MySQL and worker	41
37 Vercel, same-origin API and cookies	42
38 CI, release and rollback	43
39 Backup, restore and operations	44
40 Requirement-based test matrix	45
41 Troubleshooting in a useful order	46
42 Demonstration and completion criteria	47
A Snippet index and implementation exercises	48
B Official references and version checks (1)	49
C Official references and version checks (2)	50
D WhatsApp updates, return reminders and staff escalation	51

00 How to use this handbook

This is a self-build implementation route for an individual FYP. You create files, complete business logic, run tests and deploy the system. The handbook supplies sequencing, contracts, representative code and acceptance criteria. It is not a complete executable repository, and the examples have not been integrated with your hardware or cloud accounts.

Baseline: 20 September 2026. Development assumes Windows and PowerShell; the Sui CLI may run separately in WSL Ubuntu. UI copy can be Bahasa Melayu first with an English option. All example identities and patient data must be synthetic. This English edition follows the Chinese handbook's chapter numbering and technical decisions; wording is adapted for clarity.

Work through each stage

1. Read the objective and prerequisites. Create files at the specified project-relative paths.
2. Replace values written as <...>. Structural snippets and pseudocode need imports, types and implementation before compilation.
3. Complete the exercises and acceptance checks. Diagnose one layer at a time instead of simultaneously changing UI, database and hardware.
4. Save test requests, responses, screenshots and explanations. Commit each completed milestone.

Requirements and precedence

The primary workflow is your original Mermaid flowchart and sequence diagram, supported by the planning documents. Earlier Archify diagrams are summaries, not evidence that every retry and error branch is covered. Your later red-and-white, approximately 90% square-container requirement supersedes the older rounded UI specification.

Deliver mobile registration, staff verification and card issue/return, administrator inventory and reporting, a local reader agent, server-enforced rules, atomic database updates and Sui Testnet audit proofs. Actual hospital integration and physical door unlocking require approved external interfaces. A mock is not a completed hospital integration.

01 Roadmap and milestones

Build small demonstrable outcomes. Expose database, deployment and reader risks before investing heavily in UI polish.

Stage	Chapters	Exit outcome
A: Foundation	03-09	Repository, database, migrations, healthy API and first cloud connection
B: Business core	10-18	Authentication, registration, MRN adapter, verification, issue and return
C: User interface	19-23	Mobile registration and complete staff/admin interaction states
D: Hardware	24-28	Real card read, authenticated upload and deployed scan workflow
E: Blockchain	29-34	Move tests, Testnet package, recoverable outbox and proof verification
F: Release	35-42	Deployment, CI, recovery rehearsal and UAT evidence

Recommended sequence

First use synthetic data, a clearly labelled mock reader and mock hospital adapter. Replace the reader while preserving the ScanResult contract. Add Sui after database rules work: stopping the worker must not break an already committed registration or issue operation. Finally test real phones, the counter computer and cloud services together.

Rebuild from a clean database weekly. In the first week, prove that database data survives a restart, a phone opens the deployed HTTPS registration page, and the Windows reader returns a stable UID for the same test card. A phone's localhost is not your development PC.

Acceptance: explain where every component runs and why the USB reader program is not deployed to Railway.

02 Architecture and explicit decisions

React runs in the browser; Spring Boot owns business rules; MySQL stores operational state. A reader agent runs on the counter PC. A TypeScript worker publishes audit commitments to Sui Testnet. Visitors and counter staff do not need wallets.

TEXT / LINE NUMBERS ARE NOT SOURCE

```

1 Visitor / Staff browser -> Web UI -> /api -> Spring Boot
2                               |
3                               v
4 Local USB reader -> Reader Agent -> HTTPS -> MySQL
5                               |
6                               audit outbox (pending)
7                               |
8                               Sui worker -> Testnet

```

The diagram is a logical summary: the agent calls Backend HTTPS endpoints; it never connects directly to MySQL.

Record three architecture decisions

ADR-001, anchoring: the original sequence waits for Sui before success. This handbook recommends the existing plan's asynchronous outbox: commit the business transaction, permit handover and display Pending proof separately. If synchronous anchoring is mandatory, introduce reservation, recovery and compensation states; never hold a database lock while waiting for a blockchain response.

ADR-002, scanning: WebApp still initiates a scan logically, but the deployed implementation uses a cloud Scan Job claimed by an outbound local agent. This avoids relying on browser access to localhost, local certificates and browser-specific permission policies. Document this implementation choice explicitly.

ADR-003, category: a UID does not encode PENJAGA or other business categories. The primary hardware exercise uses pre-provisioned test-card category data. If an existing hospital card cannot be read or written, an approved UID-to-category inventory mapping is an alternative requirement decision, not an excuse to guess a category.

Issued means an allocated physical pass. It does not prove that a door controller has granted access. Create docs/adr/ entries describing context, alternatives and consequences. Acceptance: business completion and proof confirmation are driven by separate fields.

03 Install the Windows toolchain

Install Git, JDK 21, Node.js 24 LTS, an IDE, Docker Desktop and a database client. Node 24 is the selected version line; check actual Vite and SDK engine requirements. In Spring Initializr select a stable released Boot version, Java 21, Maven and Jar, not a milestone or snapshot. See R01-R04.

1. Point the IDE Project SDK and JAVA_HOME to the same JDK. Use the repository's Maven wrapper.
2. Reopen the terminal after installing Node. If PowerShell blocks npm.ps1, use npm.cmd rather than weakening execution policy globally.
3. Follow Docker's Windows prerequisites and enable its supported WSL 2 backend. Wait for the engine to start.
4. Do not use the MySQL root account for normal application access.

POWERSHELL / LINE NUMBERS ARE NOT SOURCE

```
1 git --version
2 java -version
3 node --version
4 npm.cmd --version
5 docker version
6 docker compose version
```

docker version should report both client and server. A client-only result often means the engine is unavailable. Record OS, JDK, Boot, Node, MySQL image, Sui CLI and SDK versions in docs/toolchain.md. Commit pom.xml, Maven wrapper, package-lock.json and Move.lock. Initial generation may use latest; repeatable builds should use locked dependencies and npm ci.

Acceptance: checks work in a new terminal. Kubernetes, Kafka and Redis are unnecessary at this stage.

04 Establish the project structure

Keep the existing planning directory and proposal. Each application module owns its dependencies; a monorepo build framework is unnecessary initially. The tree is a target, not a demand to implement every directory immediately.

TEXT / LINE NUMBERS ARE NOT SOURCE

```

1 FYP Dev/
2   frontend/      React + TypeScript + shadcn/ui
3   backend/       Spring Boot + Maven wrapper
4   reader-agent/  Java 21; Windows native PC/SC
5   sui-worker/    Node + TypeScript + MySQL client
6   move/hsaas_audit/ Move.toml, sources/, tests/
7   infra/         compose.yaml, deployment notes
8   docs/
9     adr/         decisions and trade-offs
10    api/         endpoint contracts, sample requests
11    evidence/    synthetic test evidence
12    runbooks/    deploy, backup, restore, reader install
13    planning/    existing plans and diagrams
14    .gitignore
15    README.md

```

GITIGNORE / LINE NUMBERS ARE NOT SOURCE

```

1 **/node_modules/
2 **/target/
3 **/dist/
4 **/.env
5 **/.env.*
6 !**/.env.example
7 **/*.keystore
8 **/secrets/
9 **/logs/
10 **/build/

```

Create the directories in the existing workspace. Inspect git status; run git init only if no repository exists. The README must explain startup order: MySQL, Backend, Frontend; Agent when scanning is required; Worker when anchoring is required.

The ignore example excludes dependencies, builds, environment secrets and logs while retaining safe .env.example files. Add project-specific paths for database backups, visitor exports, mnemonic phrases, device tokens, keystores and IDE HTTP-client secrets. Ignoring a file does not remove secrets already committed to Git.

Acceptance: git status contains only intended changes, and no environment example contains usable credentials.

05 Initialize Spring Boot

Generate at <https://start.spring.io/> using Maven, Java, a stable Boot release, Jar and Java 21. Use group edu.upm.hsaas, artifact visitor-pass and package edu.upm.hsaas. Extract directly into backend/ so backend/pom.xml exists without an extra nested project directory.

Dependency	Purpose
Spring Web / Web MVC	REST endpoints and JSON request/response handling
Validation	Required fields and length/format checks
Spring Data JPA	Entities, repositories and transactions
MySQL Driver	JDBC connectivity
Flyway Migration	Versioned schema changes
Spring Security	Authentication, roles, sessions and CSRF
Actuator	Minimal health information

Boot starter organization can differ across major versions. Trust the generated dependency management rather than copying a complete old tutorial pom. Check for Flyway's MySQL support module, commonly org.flywaydb:flyway-mysql, under the chosen Boot version. See R01 and R05.

POWERSHELL / LINE NUMBERS ARE NOT SOURCE

```
1 cd backend
2 .\mvnw.cmd -v
3 .\mvnw.cmd dependency:tree
4 .\mvnw.cmd test
```

The wrapper selects Maven; dependency:tree shows resolved versions. Context tests can fail before datasource configuration exists: finish chapters 06-07, then require passing tests rather than removing database dependencies. Create config, auth, registration, card, assignment, audit, reader, hospital and common packages beneath the main application's package. Start without Lombok so constructor injection remains explicit.

Acceptance: dependencies resolve, and Maven and the IDE both use Java 21.

06 Local MySQL and persistence

Create `infra/compose.yaml` for development only. Supply two different passwords through an untracked `infra/.env`. The application account is separate from root; root is for administration.

YAML / LINE NUMBERS ARE NOT SOURCE

```

1  services:
2    mysql:
3      image: mysql:8.4
4      environment:
5        MYSQL_DATABASE: hsaas
6        MYSQL_USER: hsaas_app
7        MYSQL_PASSWORD: ${DB_PASSWORD}
8        MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
9      ports:
10       - "127.0.0.1:3306:3306"
11     volumes:
12       - hsaas_mysql:/var/lib/mysql
13 volumes:
14   hsaas_mysql:

```

1. Create `.env` and a safe `.env.example` containing names and placeholders only.
2. From the repository root run `docker compose -f infra/compose.yaml up -d`. Explicitly use `--env-file infra/.env` if your invocation does not resolve that file; verify substitution without printing secrets.
3. Inspect service logs and wait until MySQL is ready.
4. Connect to `127.0.0.1:3306/hsaas` as `hsaas_app`. If 3306 is occupied, map host port 3307 and update JDBC configuration.

A named volume preserves data across container recreation. Do not casually use `down -v`: it deletes the associated volumes. `MYSQL_*` initialization values usually apply only to an empty data directory; editing `.env` does not change an existing database account's password.

Use `utf8mb4`. Store business timestamps in UTC; render in `Asia/Kuala_Lumpur`. Align Java Instant, MySQL UTC `DATETIME(6)` and worker timestamp serialization. Acceptance: a temporary record survives restart; subsequent schema changes go through migrations. Do not expose local 3306 to the LAN.

07 Configure Backend and the first endpoint

Put application.yml in backend/src/main/resources/. Spring Boot does not automatically load arbitrary .env files: set DB_URL, DB_USER and DB_PASSWORD in the IDE run configuration or the launching process.

```
YAML / LINE NUMBERS ARE NOT SOURCE
 1 server:
 2   port: ${PORT:8080}
 3 spring:
 4   datasource:
 5     url: ${DB_URL}
 6     username: ${DB_USER}
 7     password: ${DB_PASSWORD}
 8     hikari:
 9       maximum-pool-size: 5
10   jpa:
11     hibernate:
12       ddl-auto: validate
13     open-in-view: false
14   flyway:
15     enabled: true
16 management:
17   endpoints:
18     web:
19       exposure:
20         include: health
```

A local JDBC URL can be jdbc:mysql://127.0.0.1:3306/hsaas?connectionTimeZone=UTC. Configure database TLS according to the actual server; disabling TLS or blindly enabling public-key retrieval is not a universal cloud fix.

1. Add the first Flyway migration, then matching entities.
2. Implement GET /api/public/ping returning a small status JSON. Permit only required public routes.
3. Start with .\mvnw.cmd spring-boot:run and request the ping endpoint.
4. Check /actuator/health without exposing environment, beans, database addresses or detailed exceptions.

ddl-auto: validate detects mapping mismatches rather than silently altering tables. open-in-view: false requires services to prepare DTOs before returning, preventing hidden database access during response serialization.

Acceptance: HTTP 200, successful migration and a working datasource. Incorrect credentials must fail without revealing their values in logs.

08 Data model and migration order

This handbook calls physical inventory cards and borrowing records `card_assignments`; older plans use `passes` and `pass_assignments`. Keep registration, inventory, borrowing history and audit events separate.

Table	Minimum responsibilities
<code>users</code>	Username, password hash, active flag, role and timestamps
<code>visitor_categories / destinations</code>	Codes, display labels and enabled state
<code>visitor_registrations</code>	Random reference, category, form data, MRN result and reviewer
<code>cards</code>	Unique normalized UID, category, state and version
<code>card_assignments</code>	Card/registration, issue/due/return/close timestamps and actors
<code>audit_events / audit_outbox</code>	Immutable snapshots and separately mutable delivery state
<code>scan_jobs / scan_results</code>	Device, operation context, expiry and scan evidence
<code>idempotency_records</code>	Actor, operation, key, request hash and original result

Create V1 identity/reference tables, V2 registrations/cards, V3 assignments/constraints, V4 audit/outbox and V5 reader/idempotency tables under `backend/src/main/resources/db/migration/`. Add indexes for state, registration time, UID, `due_at` and eligible outbox work.

Never modify a migration already applied to a shared environment. Use a new migration. Do not merge people merely because their names match. Hospital approval is still required for personal-data fields and retention rules.

Acceptance: a clean database initializes, a second startup does not duplicate seeds, and an older dataset upgrades successfully. Preserve migration logs and the ERD.

09 Constraints and application layering

MySQL does not offer PostgreSQL-style WHERE partial unique indexes. A generated column can enforce one open assignment per card; NULL permits multiple closed historical rows. This example assumes BIGINT identifiers and closed_at IS NULL for active assignments.

SQL / LINE NUMBERS ARE NOT SOURCE

```
1 ALTER TABLE card_assignments
2   ADD active_card_id BIGINT GENERATED ALWAYS AS
3     (CASE WHEN closed_at IS NULL THEN card_id ELSE NULL END)
4     STORED,
5   ADD UNIQUE KEY uq_active_card (active_card_id);
```

TEXT / LINE NUMBERS ARE NOT SOURCE

```
1 registration/
2   RegistrationController.java    HTTP + @Valid
3   RegistrationService.java      rules + transactions
4   RegistrationRepository.java   database access
5   Registration.java             JPA entity
6   dto/CreateRegistrationRequest.java
7   dto/RegistrationResponse.java
8   common/error/                safe API errors
```

Add the equivalent active registration constraint, foreign keys, NOT NULL requirements and valid-state constraints. Test with actual MySQL: H2 is not an equivalent concurrency or SQL compatibility check. See R06.

Controllers handle HTTP and validation, services enforce rules and transaction boundaries, repositories handle persistence, and DTOs control the API surface. Repositories must not call Sui. Do not serialize entire entities containing internal or sensitive fields.

After ping and migrations work, complete an early deployment rehearsal using chapters 35-37. Verify Java startup, the platform PORT, private MySQL connectivity and frontend API forwarding using synthetic data.

Acceptance: inserting a second active assignment for the same card fails; closing the first allows another. A phone can reach the cloud ping endpoint.

10 Define API contracts before pages

Document endpoint, role, request, response and error codes in docs/api/. These are teaching contracts; final fields depend on approved forms. Every mutation is validated again on the server.

Endpoint	Access and responsibility
GET /api/public/registration-config	Anonymous form configuration
POST /api/public/registrations	Create a submitted registration
GET /api/staff/registrations	Authorized, bounded and paginated queue
POST /api/staff/registrations/{id}/verify	Record verification
POST /api/staff/registrations/{id}/reject	Reject with a reason
POST /api/staff/assignments	Issue using registrationId and scanResultId
POST /api/staff/assignments/{id}/return	Close assignment and release inventory
POST /api/admin/cards	Register physical inventory from a scan
GET /api/admin/proofs	Audit delivery and verification information

JSON / LINE NUMBERS ARE NOT SOURCE

```

1 {
2   "code": "CARD_CATEGORY_MISMATCH",
3   "message": "The card category is not compatible.",
4   "correlationId": "a-random-request-id",
5   "fieldErrors": []
6 }
```

Use 400 for malformed input, 401 for missing authentication, 403 for denied access or CSRF, 409 for state conflicts and 422 for business validation where that convention is adopted. Keep codes stable and add correlation IDs; do not return stack traces or SQL.

Implement a central exception handler, error-code enum, pagination limits and a sort-field allowlist. Generate OpenAPI using a springdoc version compatible with the chosen Boot release. Never concatenate arbitrary client sort fields into SQL.

Acceptance: save successful and failing requests in an HTTP client. Responses must not leak password hashes, database addresses or unnecessary audit internals.

11 Authentication and roles

Use server-side sessions, HttpOnly cookies and CSRF. Browser requests stay on the frontend's /api origin using a development proxy and production rewrite. This avoids implementing your own JWT refresh/rotation scheme; the old plan's JWT_SECRET is not used on this route.

1. Store PasswordEncoder output, never plaintext. Load account state and roles through UserDetailsService.
2. Configure Spring Security formLogin processing at /api/auth/login; send application/x-www-form-urlencoded credentials.
3. Return minimal JSON on success and 401 JSON on failure, not HTML redirects.
4. Implement GET /api/auth/me and POST /api/auth/logout; logout invalidates the session and clears the cookie.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 // Structure inside SecurityFilterChain configuration
2 http.authorizeHttpRequests(auth -> auth
3     .requestMatchers("/api/public/**", "/api/auth/csrf",
4         "/actuator/health").permitAll()
5     .requestMatchers("/api/admin/**").hasRole("ADMIN")
6     .requestMatchers("/api/staff/**")
7         .hasAnyRole("COUNTER_STAFF", "ADMIN")
8     .anyRequest().authenticated());
```

Path rules are the first boundary, not complete object-level authorization. The snippet still needs login/logout configuration, JSON handlers, session policy and a separate device-authentication filter chain.

Bootstrap the initial administrator through a one-time command/profile with an environment-supplied password only when users is empty. Remove the bootstrap secret afterwards. Never seed a fixed plaintext password. Protect the last active administrator and invalidate or refresh existing sessions after role changes.

Acceptance: anonymous staff access gets 401, staff access to admin APIs gets 403, and disabled accounts cannot log in.

12 Session, CSRF and browser requests

Expose a permitted CSRF endpoint using Spring Security's `CsrfToken` argument. Do not log returned tokens. Keep default CSRF protection and send the returned `headerName` on modifying requests.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 @GetMapping("/api/auth/csrf")
2 Map<String, String> csrf(CsrfToken token) {
3     return Map.of("headerName", token.getHeaderName(),
4                   "token", token.getToken());
5 }
```

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```
1 const csrf = await fetch('/api/auth/csrf', {
2   credentials: 'include'
3 }).then(r => r.json());
4
5 await fetch('/api/auth/login', {
6   method: 'POST', credentials: 'include',
7   headers: { [csrf.headerName]: csrf.token },
8   body: new URLSearchParams({ username, password })
9 });
```

Check `response.ok`: a resolved fetch with 401 is not successful authentication. Refresh CSRF after login and logout, clear identity caches, and include the token on POST/PATCH/DELETE, including anonymous registration submissions. See R07.

Production cookies should be `HttpOnly`, `Secure`, `Path=/` and `SameSite=Lax`, normally without a `Domain` attribute. The local HTTP profile can disable `Secure`. Debug proxy and forwarded-HTTPS handling rather than disabling CSRF to suppress 403 errors.

In-memory sessions are acceptable for an initial single-instance prototype but disappear on restart. For restart persistence or multiple instances, configure Spring Session JDBC, manage its official version-compatible schema with Flyway and test expiry cleanup.

Acceptance: missing-CSRF writes fail, correct tokens work, logged-out sessions cannot issue cards and JavaScript cannot read the session cookie.

13 Implement four registration categories

Use EXECUTIVE, PENJAGA, VENDOR and CONTRACTOR as stable codes; translate labels in the UI. Suggested name, phone, organization and purpose fields are not a substitute for hospital-approved forms.

1. Return enabled categories, destinations, field rules and a version from the public configuration API.
2. Define common and category-specific request fields. Use Bean Validation for format/length and service checks for conditional rules.
3. For PENJAGA require a current verification bound to this MRN and ward. Reject unnecessary patient fields for other categories.
4. Generate a random public reference, save SUBMITTED and return the minimum public acknowledgement.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 // DTO fragment: add your validated category-specific fields
2 public record CreateRegistrationRequest(
3     @NotBlank @Size(max = 120) String fullName,
4     @NotNull VisitorCategory category,
5     @NotNull Long destinationId,
6     @NotBlank @Size(max = 32) String phone,
7     String mrnVerificationToken
8 ) {}
```

A request DTO is not an entity. It must not let anonymous users assign verifiedBy or VERIFIED status. Store phone numbers as strings to preserve prefixes and leading zeros. Validate identity documents according to document type rather than imposing one national format on every visitor.

Implement consent/notice copy, field allowlists, body-size limits, retry keys, rate limits and field error mapping. A random reference reduces guessing but does not authorize access to private details.

Acceptance: test four valid categories, missing/oversized fields, unknown categories, disabled destinations, duplicate submissions and forged status fields.

14 Hospital API and MRN validation

The hospital's API contract and credentials are not confirmed. Define an adapter first and explicitly separate mock and real implementations. Return VALID, INVALID or UNAVAILABLE: a timeout is not proof that a patient does not exist.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 public interface HospitalGateway {
2     MrnCheckResult validate(String mrn, String wardCode);
3 }
4 // MrnCheckResult is your own result DTO.
5 // Implement MockHospitalGateway first.
6 // Implement ApprovedHospitalApiGateway after receiving a contract.
```

1. Use deterministic fixtures for matching, mismatching and timeout outcomes. Do not accept every non-empty MRN.
2. Set connection/response timeouts and log correlation IDs rather than patient payloads.
3. Store validation evidence and issue a short-lived token bound to the session/nonce, MRN fingerprint, ward, result and expiry.
4. Recheck the token during final registration. Changing MRN or ward invalidates the earlier result regardless of the UI's green checkmark.
5. Replace the adapter only after agreeing authentication, fields, status codes, limits, minimal data and a test environment. Add contract tests.

The original flow requires successful validation. Default to strict handling: INVALID asks for correction; UNAVAILABLE preserves input and offers retry. An approved manual-review alternative must remain MRN_PENDING until recorded review is complete; never silently convert an outage into VALID.

HOSPITAL_MODE=mock|real is a backend concern. Never expose hospital credentials in the browser.

Acceptance: demonstrate all three outcomes, changed-input rejection and a visible mock-mode label without enabling public patient enumeration.

15 Queue, verification and rejection

SUBMITTED and VERIFIED are different states. Only verified registrations may receive an assignment. Start with a bounded queue refreshed about every five seconds; do not introduce WebSockets before the requirement warrants them.

Implement paginated queue and authorized detail endpoints. Mask unnecessary contact/document details in lists. Verification is a transaction that checks current state and MRN rules and records actor, timestamp, policy version and audit event. Rejection requires a reason and an eligible state; an issued registration cannot simply be overwritten as rejected.

TEXT / LINE NUMBERS ARE NOT SOURCE

```
1 SUBMITTED -> VERIFIED
2 SUBMITTED -> REJECTED
3 VERIFIED -> assignment allowed (if no active assignment)
4 REJECTED -> no assignment
```

Derive the actor from authentication, not request JSON. Handle competing reviewers with an explicit already-processed result or conflict. The UI should show visitor details, category, destination and MRN source before review, and enable assignment only after successful verification.

Acceptance: directly calling issue with SUBMITTED or REJECTED fails. New submissions appear within the agreed refresh interval. Simultaneous reviewers do not overwrite each other's audit evidence.

16 Inventory and the issue transaction

A scan means that a device reported bytes in a particular context. Backend decides eligibility. Admin submits scanResultId; Backend validates category, normalizes UID and inserts the inventory record. A unique database constraint is required even when an earlier lookup reported no duplicate.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 // Service structure; helper methods are for you to implement.
2 @Transactional
3 public AssignmentResponse issue(IssueCommand command, Actor actor) {
4     var registration = registrations.lock(command.registrationId());
5     var scan = scans.lockUsable(command.scanResultId(), actor);
6     var card = cards.lockByUid(scan.uid());
7     rules.requireVerifiedAndCompatible(registration, card, scan);
8     var assignment = assignments.create(registration, card, actor);
9     card.markIssued();
10    scans.consume(scan);
11    audit.appendAndEnqueue(assignment, actor);
12    return mapper.toResponse(assignment);
13 }
```

@Transactional makes registration/scan/card checks, assignment creation, inventory state, scan consumption and audit/outbox writes one atomic unit. Implement lock methods using JPA PESSIMISTIC_WRITE or SELECT FOR UPDATE. Require VERIFIED, no active assignment, a registered AVAILABLE card, compatible category and a fresh scan belonging to the actor and operation.

consume prevents reusing the same scan. appendAndEnqueue writes locally in the same transaction; it must not wait for a network request. Define one lock order across issue, return and background commands. Database uniqueness remains the final concurrency guard.

Acceptance: two staff issuing the same card to different visitors produce exactly one success. Injecting an audit-write failure rolls back assignment, inventory, scan consumption and outbox together.

17 Idempotency, return and overdue

Disabling a button cannot make network retries safe. Use an Idempotency-Key scoped by actor and operation. A reused key with a different request hash returns 409.

Generate a UUID once per user command and retain it for retries. Claim the idempotency record in the business transaction and save the original resource/result. Completed identical requests return that result; in-progress conflicts have an explicit retry contract. A rolled-back transaction must not leave a false completion record.

TEXT / LINE NUMBERS ARE NOT SOURCE

```
1 Card AVAILABLE -> ISSUED -> AVAILABLE
2               ^
3 Card ISSUED -> OVERDUE -----+ (returned)
4 Card ISSUED / OVERDUE -> LOST (authorised action)
5 Card AVAILABLE -> DISABLED (administration)
```

Return locks the relevant records, sets returnedAt/returnedBy/closedAt, releases the active-assignment uniqueness constraint and appends a RETURNED event/outbox record. RETURNED is an event; the inventory becomes AVAILABLE. If a card cannot be read, any manual lookup path requires authorization and a reason, not fabricated scan evidence.

The overdue scheduler processes due_at before UTC now in batches. Lock and recheck that the assignment remains open. Make the overdue event unique per assignment and retain the due-time policy snapshot from issuance; later settings changes must not silently alter old deadlines.

Acceptance: repeated return produces no duplicate event, returned cards can be issued again, and concurrent overdue/return processing leaves consistent state. Inject a Clock for time-dependent tests.

18 Audit, reporting and administration

Keep operational state, immutable event snapshots and mutable proof-delivery state separate. Pending proof does not mean card issue failed.

Implement account creation/disable/role changes with last-admin protection; inventory lost/disabled actions without deleting active cards; versioned destinations/rules; dashboard counts; and a filtered audit viewer showing Pending, Confirmed and Failed independently of business state.

SQL / LINE NUMBERS ARE NOT SOURCE

```
1 SELECT c.code, COUNT(*) AS registrations
2 FROM visitor_registrations r
3 JOIN visitor_categories c ON c.id = r.category_id
4 WHERE r.registered_at >= :startUtc
5       AND r.registered_at < :endUtc
6 GROUP BY c.code;
```

Bind report parameters. Half-open UTC ranges avoid double counting across dates. Derive historical metrics from registration, assignment and event timestamps, not a sum of today's inventory state. Convert site-local reporting boundaries to UTC consistently.

Require report authorization, mask exports by default and neutralize spreadsheet-formula prefixes such as =, +, - and @ in CSV cells. Use the same filter conditions for tables and totals.

Acceptance: a fixed fixture of three registrations in each of four categories totals twelve everywhere. Perform any deliberate tampering experiment only in an isolated test database.

19 Initialize React, Vite and shadcn/ui

Create frontend/ from the repository root. First get the default React/TypeScript page working, then add styling and components. Follow the Tailwind Vite-plugin route rather than mixing Tailwind 3 instructions with the newer setup. See R02 and R08.

POWERSHELL / LINE NUMBERS ARE NOT SOURCE

```
1 npm.cmd create vite@latest frontend -- --template react-ts
2 cd frontend
3 npm.cmd install
4 npm.cmd install tailwindcss @tailwindcss/vite
5 npm.cmd install -D @types/node
```

POWERSHELL / LINE NUMBERS ARE NOT SOURCE

```
1 npx.cmd shadcn@latest init
2 npx.cmd shadcn@latest add button input label select
3 npx.cmd shadcn@latest add table dialog alert badge tabs
4 npx.cmd shadcn@latest add sheet skeleton sonner
5 npm.cmd install react-router-dom @tanstack/react-query
6 npm.cmd install react-hook-form zod @hookform/resolvers
7 npm.cmd run dev
```

Configure React and Tailwind plugins in vite.config.ts. Set the @ alias in both TypeScript configuration and Vite resolution; the CLI needs consistent paths. Import the generated CSS from main.tsx and retain its semantic tokens. Add a real type-check/build step and commit package-lock.json.

Each CLI command generates editable component source, not an entire application. Review generated files, then compose buttons, inputs, labels, tables, dialogs and alerts into your flows. Do not copy paid template assets without permission.

Acceptance: the default page builds, a shadcn button renders with styling, and keyboard focus is visible. A successful package installation alone is not UI verification.

20 Frontend structure and request layer

Separate shared UI from feature-specific forms and state. Keep API access centralized instead of scattering raw fetch logic across every component.

TEXT / LINE NUMBERS ARE NOT SOURCE

```
1 frontend/src/
2   app/           router.tsx, providers.tsx
3   components/ui/ shadcn generated components
4   components/    PageHeader, StatusBadge, ScanPanel
5   features/
6     auth/        login, session hooks
7     registration/ mobile forms, queue, review
8     cards/        inventory, add-card flow
9     assignments/  issue, return, overdue
10    audit/        proof list, verifier
11    lib/          api.ts, errors.ts, dates.ts
12    styles/       tokens.css
```

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```
1 // Add inside Vite's defineConfig object.
2 server: {
3   proxy: {
4     '/api': { target: 'http://localhost:8080', changeOrigin: true }
5   }
6 }
```

Add a query/cache provider, router and localized labels. Suggested routes include `/register`, `/register/success`, `/staff/registrations` and `/admin/*`. Protected routes improve UX, but Backend still authorizes every request.

The API client handles credentials, CSRF, JSON parsing, `response.ok` and the common error shape. A 401 clears identity state and routes to login without looping; a 409 displays the business conflict. Mutation success invalidates affected queue, inventory and assignment queries.

The Vite proxy is development-only. Production requires the chapter 37 rewrite or a separately designed cross-origin architecture. Do not embed secrets in `VITE_*` variables; these are exposed to the browser bundle.

Acceptance: frontend and API work together through `/api`, failure responses stay failures, refresh works on deep links, and cached state updates after a mutation.

21 Red-and-white square UI system

Use a white workspace, dark red primary actions and approximately 90% square containers/table surfaces. Avatars and small status dots may remain round. Do not make every surface red: success, pending and errors need distinct text and visual treatment.

CSS / LINE NUMBERS ARE NOT SOURCE

```
1  :root {
2    --background: #fafafa;
3    --foreground: #171717;
4    --card: #ffffff;
5    --primary: #9d0b0f;
6    --primary-foreground: #ffffff;
7    --border: #e5e5e5;
8    --radius: 0rem;
9  }
10 @theme inline {
11   --radius-sm: 0px;
12   --radius-md: 0px;
13   --radius-lg: 0px;
14   --radius-xl: 0px;
15 }
```

Adjust the semantic tokens generated by shadcn rather than deleting unrelated ones. New versions can add radius tokens, so inspect the actual generated CSS. Avoid a universal border-radius:0 rule that breaks avatars and radios. These colors are design suggestions, not approved hospital brand specifications.

Build AppShell, PageHeader, FilterBar/Table and ScanPanel in that order. Prefer dividers and clear spacing over deeply nested cards. Each page needs a primary action and meaningful empty/loading/error/disabled states.

Use a single-column mobile registration layout around 390px and a queue/detail workspace around 1440px. Aim for approximately 44px touch targets, associated labels, visible focus and adequate contrast. Dialogs must manage focus and return it on close; color cannot be the only error signal.

Acceptance: create a component showcase and test all interaction states. shadcn Studio and NameThatUI are visual references; do not copy licensed assets directly.

22 Mobile registration from QR to reference

The QR contains the public HTTPS registration URL and a non-sensitive counter identifier, not patient data or a staff token. Validate the counter identifier server-side.

Load configuration, choose a category, then render its fields. PENJAGA requires MRN/ward validation before continuing. Changing either field clears the validation state. Preserve form input when the API is unavailable; display mock mode when using fixtures.

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```
1 // UI validation fragment; server repeats these rules.
2 const commonSchema = z.object({
3   fullName: z.string().trim().min(1).max(120),
4   phone: z.string().trim().min(7).max(32),
5   category: z.enum([
6     'EXECUTIVE', 'PENJAGA', 'VENDOR', 'CONTRACTOR'
7   ])
8 });
```

React Hook Form and a schema library can supply client feedback, but they do not replace backend conditional validation. During submit, use an idempotency key and disable duplicate clicks. On failure restore interaction without clearing the form. On success show only the reference and next-step instructions, not internal identifiers or patient details.

Implement distinct configuration-loading, validation-in-progress, invalid-MRN, unavailable-hospital, submission-failed and submitted states. Protect against stale validation responses after fields change.

Acceptance: complete all four categories on a phone, retry after a temporary outage and verify the browser cannot bypass server rules by editing requests.

23 Staff/admin UI and scan state

Build login, queue, review, assign, return/overdue, inventory/add-card, user/settings, audit/verifier and analytics views. Every data view needs explicit loading, empty and error states.

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```
1 type ScanState =  
2   | { kind: 'idle' }  
3   | { kind: 'waiting'; jobId: string }  
4   | { kind: 'read'; scanResultId: string; uid: string }  
5   | { kind: 'error'; code: string; canRetry: boolean }  
6   | { kind: 'submitting'; idempotencyKey: string }  
7   | { kind: 'done'; assignmentId: string };
```

Use a reducer or similarly explicit state machine for scanning. Creating a job, waiting for a tap, obtaining a result and submitting an assignment are different steps. Disable competing actions only where necessary and keep failures visible.

The assignment panel displays the verified registration, category, device status and current job. Cancel/timeout closes that job; retry creates a new one. Ignore late responses from older job IDs. Never silently reuse the previous UID after a read failure.

Show unregistered, unavailable, category mismatch and reader failure as separate messages. After success invalidate queues and inventory, show the assigned UID, then offer handover confirmation as required by the workflow. Audit Pending is not an issue error.

Acceptance: unplug, timeout, cancel, retry, mismatched card, duplicate submission and late response all lead to understandable, recoverable UI states.

24 Hardware preparation and compatibility

The hardware choice is currently only 'NFC reader and cards'; no exact model has been confirmed. Use Windows plus a USB PC/SC reader as the baseline. ACS ACR1252U is a documentation reference, not proof that a purchased device or existing hospital card will work.

Check USB interface, Windows drivers, PC/SC support, NFC frequency/protocol, card technology, UID behavior and writable memory. A 125 kHz RFID card is not automatically compatible with a 13.56 MHz NFC reader. Match the card and reader before purchasing. See R09-R11.

For the primary exercise, use test cards with an agreed readable category profile. The UID identifies a card record; it does not contain a visitor category, establish the holder's identity or resist cloning by itself. Do not alter real hospital cards or security keys.

If category memory is inaccessible, document an approved UID inventory mapping alternative. In that mode the administrator assigns category during enrollment and the server retrieves it on later scans. Do not present this as reading category from the card.

Acceptance: record hardware model, firmware/driver, card type, UID length and byte order, and category source in docs/hardware-matrix.md.

25 Windows drivers and independent reading

Test with the manufacturer's utility before implementing Java. The agent runs on native Windows; containers, WSL and Railway do not automatically see the counter's USB device.

1. Install the matching vendor driver and confirm the reader appears in Device Manager.
2. Check the Windows Smart Card service. Close competing utilities that hold an exclusive connection.
3. Select the contactless/PICC slot, not an unrelated SAM slot.
4. Use the model's documented UID command and record response data and status separately.
5. Repeat tap/remove twenty times to check UID consistency and removal behavior.

JAVA / LINE NUMBERS ARE NOT SOURCE

```
1 // Diagnostic fragment, inside a method; Java 21.
2 var terminals = TerminalFactory.getDefault().terminals().list();
3 if (terminals.isEmpty()) throw new IllegalStateException("NO_READER");
4 var terminal = terminals.get(0); // choose by configured name in agent
5 if (!terminal.waitForCardPresent(15000)) return;
6 var card = terminal.connect("");
7 try {
8     var reply = card.getBasicChannel().transmit(
9         new CommandAPDU(new byte[]{
10             (byte) 0xFF, (byte) 0xCA, 0, 0, 0});
11     if (reply.getSW() != 0x9000) throw new IllegalStateException("READ_FAILED");
12     String uid = HexFormat.of().withUpperCase().formatHex(reply.getData());
13 } finally { card.disconnect(false); }
```

Import `javax.smartcardio.*` and `java.util.HexFormat`. FF CA is a reader pseudo-APDU supported by relevant readers, not a universal command for every device. It retrieves UID, not category. `getSW()` checks 9000; `getData()` excludes the status bytes. Implement terminal selection, timeout handling and exception mapping. Never use ATR as UID.

Acceptance: vendor utility and Java agree on UID; no-reader, timeout, unplug and invalid-status cases do not return fabricated success.

26 Card provisioning and category parsing

Provisioning is preparation of test-card data, not ordinary daily enrollment. Confirm the exact card memory model and vendor commands before writing anything.

Use a dedicated test-card set. Define an NDEF record type, version and category allowlist. Write and read back with an approved utility, then test the same profile through Java. Keep raw fixture bytes for valid, unsupported-version, malformed, truncated and unknown-category cases.

JSON / LINE NUMBERS ARE NOT SOURCE

```
1 {  
2   "v": 1,  
3   "profileId": "random-card-profile-id",  
4   "category": "PENJAGA"  
5 }
```

NDEF framing, TLV parsing, capacity and read-page commands depend on the card technology. Do not invent a universal category APDU or treat an arbitrary byte offset as a complete NDEF parser. Never modify lock bytes, access keys or hospital cards in a learning exercise.

Backend validates category against its allowlist and inventory. Readable category data is not inherently authentic; an optional signed profile requires a defined canonical format, signature verification and enough memory. The registration category must still match inventory rules.

Acceptance: parser rejects malformed/unknown profiles. Normal Add Card and Assign actions read rather than rewrite category. Document UID mapping explicitly if chosen instead of card provisioning.

27 Reader Agent and cloud scan jobs

Create a Java 21 reader-agent module using `java.smartcardio` and an HTTPS client. Backend owns job lifecycle. The agent connects outbound and has no local browser-facing HTTP listener.

TEXT / LINE NUMBERS ARE NOT SOURCE

```
1 Browser -> POST /api/staff/scan-jobs
2 Backend -> save WAITING job (user, device, purpose, expiry)
3 Agent    -> POST /api/device/scan-jobs/claim
4 Agent    -> wait for physical tap; read UID + profile
5 Agent    -> POST /api/device/scan-jobs/{id}/result
6 Browser  -> GET /api/staff/scan-jobs/{id}
7 Backend  -> READY + scanResultId
```

Use a separate stateless device-authentication chain for `/api/device/**`. A high-entropy per-device token authenticates only device routes, not human sessions. Store a hash or appropriate verifier server-side, support rotation/revocation and compare safely. CSRF exemption is limited to stateless token-authenticated device routes; it does not apply to staff/public browser writes.

Bind each job to device, counter, initiating actor, purpose, target registration, nonce and expiry. Permit one active job per device. Atomically claim with a lease; accept one matching result, reject expired/cancelled/wrong-device uploads and consume evidence once during the business transaction.

The agent performs removal-before-next-scan, reads UID/profile, then uploads only the required fields and current job identifiers. Offline/failed reads produce explicit failures. The browser polls its own job and cannot read another user's evidence.

Acceptance: wrong token, wrong device, reused nonce, old result and cross-counter attempts fail. Agent has neither database credentials nor Sui keys.

28 Package and install the agent

First compile and run the reader module manually on the actual counter PC. Package with jpackage or an approved installer; include java.smartcardio if creating a reduced runtime. Keep configuration separate from secrets.

PROPERTIES / LINE NUMBERS ARE NOT SOURCE

```
1 # Local agent configuration example; token stored separately.
2 api.baseUrl=https://your-api-domain
3 device.id=counter-01
4 reader.name=YOUR_VERIFIED_PCSC_READER_NAME
5 scan.timeoutSeconds=30
6 heartbeat.intervalSeconds=10
```

Use a unique device identity/token, the Backend HTTPS URL and configured reader name. Store tokens in Windows-protected storage or an ACL-restricted location, not source control or logs. Install using a dedicated Windows account and Task Scheduler or a suitable service wrapper; test required hardware access in that account.

Provide a health/heartbeat signal to Backend, a bounded retry/backoff policy and rotating sanitized logs. A local utility may help diagnostics, but do not expose an unauthenticated listener on 0.0.0.0.

At the counter, test reboot/autostart, HTTPS connectivity, reader selection, unplug/reconnect, network loss, lease expiry, token revocation and a normal tap/remove sequence. Keep a printed recovery procedure.

Acceptance: a fresh PC can be installed from the runbook; agent recovery does not accidentally complete a cancelled or expired job.

29 What Sui proves and what it does not

Keep personal data, MRN, phone numbers, raw card identifiers and detailed audit snapshots off-chain. Sui records a commitment to an event snapshot, not the medical truth of the event or the physical identity of a cardholder.

Define proof v1 with a random eventId, payloadVersion, canonical payload and a strong random nonce. Store exact canonical bytes and their SHA-256 digest locally. Version the encoding, timestamp format, field order and null handling rather than relying on incidental JSON serializer output.

TEXT / LINE NUMBERS ARE NOT SOURCE

```
1 CanonicalAuditV1 (example ordered fields)
2 schemaVersion, eventId, eventType, occurredAtUtcMs,
3 assignmentRef, actorRef, beforeStatus, afterStatus, nonce
4
5 payloadHash = SHA-256(UTF-8(canonicalBytes))
```

Backend and worker must hash the same bytes. A hexadecimal SHA-256 string contains 64 characters but represents 32 bytes. Hashing PII without randomization does not automatically anonymize it; low-entropy data may be guessed. Do not anchor directly identifiable fields.

Treat events as append-only. Correction creates another event referring to the original, rather than rewriting history. The outbox can change delivery state while the underlying snapshot remains fixed.

Acceptance: repeated hashing of identical canonical bytes matches across implementations; changing one field produces a different digest. Explain that a matching proof shows consistency with an anchored commitment, not that the original data was accurate.

30 Sui CLI and Testnet wallets

Follow official Sui installation guidance, using WSL Ubuntu for the CLI if preferred while leaving the USB agent on Windows. Commands below are Bash, not PowerShell. See R12-R14.

BASH / LINE NUMBERS ARE NOT SOURCE

```
1 suiup install sui@testnet
2 sui --version
3 sui client
4 sui client envs
5 sui client switch --env testnet
6 sui client active-address
7 sui client gas
```

Install the Testnet-compatible CLI via the official documented method, select the Testnet environment and verify the active network/address before funding. Use <https://faucet.sui.io/> for test tokens and check the balance. Never use Mainnet funds or a personal production wallet for this FYP.

Separate a publisher/upgrade identity from the everyday writer where practical. The publisher controls package publication and retained upgrade authority; the worker holds only the capability required to anchor events. Store recovery material offline and secrets outside Git.

Record CLI version, network, addresses and public object IDs in the deployment manifest. Do not record private keys or mnemonic phrases there. Select a key format supported by the pinned SDK.

Acceptance: the selected address has Testnet gas, can be inspected on the correct network and is not the user's personal wallet.

31 Design the Move package

Create the package with `sui move new hsaas_audit` under `move/`. Inspect generated `Move.toml` and use the generated edition/dependency conventions for your CLI instead of copying stale configuration.

Design `WriterCap` for authority, a shared `Registry` storing `eventId-to-digest` commitments, and `AuditAnchored` event data. Initial publication creates the registry and transfers the capability to the intended owner. An ordinary caller without capability must not anchor.

MOVE / LINE NUMBERS ARE NOT SOURCE

```
1 // Signature sketch inside your audit module; not a full module.
2 public fun anchor(
3     cap: &AuditWriterCap,
4     registry: &mut Registry,
5     event_id: vector<u8>,
6     payload_hash: vector<u8>,
7     payload_version: u64,
8     ctx: &mut TxContext,
9 ) {
10     // Check capability-to-registry binding and input lengths.
11     // If ID exists: require the same hash, then return.
12     // Otherwise store the proof and emit AuditAnchored.
13 }
```

This is an entry-function signature, not a complete module. Implement types, initialization, table access, events and tests. Validate event ID encoding and 32-byte digest length. Same `eventId` plus same digest should be idempotent; the same `eventId` with a different digest must abort rather than overwrite.

Bind the writer capability to the intended registry/deployment if the design permits multiple registries. Explicitly decide upgrade authority, cap custody and rotation. Do not claim immutability beyond what retained upgrade powers and module design actually enforce.

Acceptance: compile, run permission/length/idempotency tests, and explain every stored field. Use the framework reference matching the actual compiler version; see R15.

32 Test and publish Move

Write tests before publication: authorized anchor succeeds, missing/wrong capability is rejected by the type/authority design, malformed inputs abort, identical retries are safe and conflicting hashes cannot replace commitments.

BASH / LINE NUMBERS ARE NOT SOURCE

```
1 # Run inside move/hsaas_audit
2 sui move build
3 sui move test
4 sui client active-env
5 sui client active-address
6 sui client publish --help
7 sui client publish
```

Run build and tests in move/hsaas_audit. Recheck active environment/address, gas and publication parameters before executing publish. Publication changes Testnet state and consumes test gas; commands in this guide are for you to execute, not actions already performed.

Record transaction digest, package ID, Registry ID, WriterCap ID and any UpgradeCap ownership from the successful result. Verify each object exists on the selected Testnet and has expected ownership/type.

Configure the worker from these actual values. An explorer screenshot alone does not prove the permission and retry behavior; preserve test output and one end-to-end synthetic event.

Acceptance: the deployment manifest agrees with the chain, and a correctly authorized transaction can anchor a test commitment without placing PII on-chain.

33 TypeScript worker and outbox

Initialize sui-worker with `npm init -y`. Add `@mysten/sui` and `mysql2`; use `typescript`, `tsx` and `@types/node` as development dependencies. Select `type=module`, NodeNext compilation, dev via `tsx`, build via `tsc` and start via `node dist/main.js`. Commit dependency locks.

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```

1 import { SuiGrpcClient } from '@mysten/sui/grpc';
2 import { Transaction } from '@mysten/sui/transactions';
3 import { Ed25519Keypair } from '@mysten/sui/keypairs/ed25519';
4
5 // env and proof are your validated config / outbox DTO.
6 const client = new SuiGrpcClient({
7   network: 'testnet', baseUrl: env.SUI_GRPC_URL
8 });
9 const signer = Ed25519Keypair.fromSecretKey(env.SUI_PRIVATE_KEY);
10 const tx = new Transaction();
11 tx.moveCall({
12   target: `${env.PACKAGE_ID}::audit::anchor`,
13   arguments: [tx.object(env.WRITER_CAP_ID),
14     tx.object(env.REGISTRY_ID),
15     tx.pure.vector('u8', proof.eventIdBytes),
16     tx.pure.vector('u8', proof.hashBytes),
17     tx.pure.u64(proof.version)]
18 });

```

The snippet constructs a transaction and assumes your chapter 31 signature. Sui supplies `TxContext`; it is not an ordinary argument. Encode event IDs consistently and convert digest hex into 32 bytes instead of submitting 64 ASCII characters.

Claim eligible work in a short database transaction using `FOR UPDATE SKIP LOCKED` plus a lease. Commit the claim before contacting Sui. A single sequential worker is sufficient initially and avoids unnecessary capability/gas contention. Give its database account only required event reads and outbox updates.

Validate network, node URL, expected signing address, cap ownership and gas at startup. Missing configuration must fail clearly. Never put private keys in `VITE_*` variables.

Acceptance: a labelled fake gateway can process test fixtures, followed by a real Testnet smoke. Never relabel a simulated response as genuine on-chain confirmation.

34 Confirmation, retry and proof checking

A resolved SDK promise is not necessarily a successful transaction. Inspect the result union/status, record the digest and wait/read for confirmation using the pinned SDK's supported methods. See R16-R18.

TYPESCRIPT / LINE NUMBERS ARE NOT SOURCE

```
1 const result = await client.signAndExecuteTransaction({
2   signer, transaction: tx, include: { effects: true }
3 });
4 if (result.FailedTransaction) {
5   throw new Error('CHAIN_EXECUTION_FAILED');
6 }
7 if (!result.Transaction) throw new Error('UNEXPECTED_RESPONSE');
8 await client.waitForTransaction({ result });
9 const digest = result.Transaction.digest;
```

Use PENDING, PROCESSING with a lease, SUBMITTED with digest, CONFIRMED, FAILED and RECONCILING as explicit delivery states. Use compare-and-set updates tied to the claim token so stale workers cannot overwrite newer outcomes.

If the response is lost after submission, do not blindly create a new business event. Reconcile the known digest or registry eventId commitment. Matching commitments confirm; mismatches require investigation; temporary node unavailability stays unknown/retryable. A Move abort remains failure. Backoff and bound retries; recover expired claims.

The verifier independently recomputes the saved snapshot hash, retrieves the intended network/registry commitment and compares it. Show MATCH, MISMATCH, NOT_ANCHORED and UNAVAILABLE separately. Never display MATCH merely because the local outbox says CONFIRMED.

Acceptance: test worker crash, lost response, expired lease, Move abort, changed snapshot and node outage. Business completion remains separate from proof delivery.

35 Deployment topology and configuration

Deploy the frontend to Vercel; Backend, MySQL and worker to Railway; agent to the counter PC; Move package to Sui Testnet. Only Backend needs a public business API. MySQL uses private networking and the worker does not need a public web port.

Location	Configuration
Frontend	Relative /api path and non-secret display settings
Backend	DB URL/user/password, profiles, sessions, hospital and device authentication
Worker	DB settings, gRPC URL, private key, expected address, package/cap/registry IDs
Reader Agent	HTTPS API URL, device identity/token and reader name
Platform	Root directories, build/start commands, PORT, health check, region and secrets

Separate local, test and demo environments with distinct databases and Testnet objects. Clearly label mock dependencies and prohibit real personal data in test fixtures. A Railway MySQL URL is not automatically a JDBC URL; Java and mysql2 require their own supported formats.

Spring's `${PORT:8080}` syntax belongs in application configuration. Railway variable references use platform syntax; do not interchange them. Check current plan pricing and limits before creating paid resources. This handbook has not created accounts, enabled billing or deployed anything for you.

Acceptance: identify each secret's owner and explain why frontend, worker and reader cannot access unrelated credentials.

36 Railway Backend, MySQL and worker

Create a test project/environment and MySQL service with persistent storage. Configure Backend's repository root as /backend and choose a supported build path or a multi-stage Dockerfile.

Use private database host information to construct JDBC configuration, supply a production-like profile and least-privilege credentials, bind the platform PORT and expose minimal /actuator/health. Inspect migration/startup logs before generating a public HTTPS domain.

DOCKERFILE / LINE NUMBERS ARE NOT SOURCE

```
1 # Dockerfile fragment: build stage must produce a known app.jar.
2 FROM eclipse-temurin:21-jre
3 WORKDIR /app
4 COPY --from=build /out/app.jar app.jar
5 USER 10001
6 ENTRYPOINT ["java", "-jar", "app.jar"]
```

The Docker example is only the runtime stage. Implement a JDK 21 build stage that copies Maven wrapper/pom/source, runs package and places a known executable Jar at /out/app.jar. Do not rely on an ambiguous target/*.jar glob. Ensure Unix wrapper permissions and LF line endings.

Test the image locally. Do not permanently skip tests to make deployment green: CI should test the same commit. Deploy the worker from /sui-worker with npm ci and npm run build, then npm start; supply secrets and do not enable a public domain.

Acceptance: Backend reconnects after restart, Worker can read the outbox, MySQL is not publicly exposed and health output contains no credentials or database details. See R19-R20.

37 Vercel, same-origin API and cookies

Import the repository with frontend as the root, Vite as framework, npm run build as build command and dist as output. Match API forwarding before the React Router fallback. Replace the destination placeholder with the actual Backend HTTPS domain.

JSON / LINE NUMBERS ARE NOT SOURCE

```
1 {
2   "rewrites": [
3     {
4       "source": "/api/:path*",
5       "destination": "https://YOUR-API-DOMAIN/api/:path*"
6     },
7     {
8       "source": "/(.*)",
9       "destination": "/index.html"
10    }
11  ]
12 }
```

Verify real proxy behavior for Set-Cookie, POST requests, error status and forwarding. Private API responses, CSRF and login must not be publicly cached; use Cache-Control: no-store.

1. Open /register and refresh a staff deep link; neither should return 404.
2. Request /api/auth/csrf and confirm JSON rather than SPA HTML.
3. Log in and inspect HttpOnly, Secure, host-only Path=/ cookies and persistent identity.
4. Submit a CSRF-protected mutation through the proxy; there must be no redirect to localhost.
5. Configure forwarded headers only for the trusted deployment path.

Preview and production must not accidentally share sensitive backends. Direct cross-origin access requires its own CORS allowlist and cookie policy, not just a different URL.

Acceptance: a real phone scans the public QR, staff use the deployed UI, and the local agent reaches Backend over HTTPS to complete registration-to-issue. See R21.

38 CI, release and rollback

Make builds reproducible from a clean clone without IDE caches or live hospital data. Use isolated test credentials and databases.

CI jobs should run Backend Maven verify against real MySQL integration tests, frontend lint/component tests/build, worker TypeScript build and fake-gateway tests, Move build/test with the pinned CLI and reader compilation/parser/mock-transport tests. Actual USB and Testnet smoke tests are explicit manual gates.

CI must not publish a new package automatically on every PR or share a mutable demo registry across unrelated tests. Record commit, artifact/image identifiers, migration version, package/registry and agent version in a release manifest.

Use expand-and-contract schema changes: add compatible fields, deploy compatible code, migrate data, then remove obsolete fields later. Rolling back application code does not undo a Flyway migration. Prefer forward fixes; use a rehearsed restore procedure when data recovery is required.

Acceptance: a new clone builds; invalid migrations stop release; compatibility and rollback conditions are documented. A release checklist covers tests, backup, migration rehearsal, secrets, deployment, smoke, observation window and fallback.

39 Backup, restore and operations

A backup is not verified until restored. Use available platform backup features and controlled logical exports as appropriate; do not assume a pricing tier includes automatic backups. Restrict access to exports and keep them out of Git.

BASH / LINE NUMBERS ARE NOT SOURCE

```
1 # Example run on an authorised machine with DB connectivity.
2 mysqldump --host=DB_HOST --user=BACKUP_USER -p \
3   --single-transaction --no-tablespaces \
4   --result-file=hsaas-backup.sql hsaas
```

This is a Bash example. `--result-file` avoids shell redirection encoding surprises; `-p` prompts for the password. `--single-transaction` primarily supports consistent InnoDB dumps while avoiding concurrent schema changes. Test the command and required privileges against the actual server.

Pause the test worker, create an independent restore database and import with an approved tool or `mysql SOURCE`. Do not overwrite the only original database. Compare counts, foreign keys, recent events, active assignments, user roles and Flyway history.

A restored outbox may lag the chain. Reconcile against the original network/registry before resuming publication. Record restore duration and the possible data-loss interval.

Log correlation IDs, anonymous resource IDs, error codes and duration, never passwords, tokens, MRN or full forms. Monitor queue age, worker/device heartbeat, failed outbox work, DB pool usage and API errors.

Acceptance: rehearse Backend restart, worker outage and reader removal, then recover using the runbook rather than memory.

40 Requirement-based test matrix

Scenario	Required evidence
Four registration categories	Different field rules; valid reference on success
Invalid MRN / API timeout	INVALID differs from UNAVAILABLE; input preserved
Read failure / no reader	Retry possible; previous UID is not reused
Duplicate UID	409 and no second inventory record
Unverified registration	Backend rejects issue even when UI is bypassed
Unregistered / unavailable / wrong category	Distinct rejection without assignment changes
Concurrent issue of one card	Exactly one active assignment
Repeated HTTP command	Original result; no duplicate event/outbox
Return and overdue race	Consistent state without duplicate event
Old / cross-counter scan	Expiry/ownership rejection
Sui lost response / outage	Reconcile or pending, not false business failure
Move abort	Failed, not Confirmed
Tampered snapshot / unavailable node	Mismatch differs from Unavailable
Database restore	Complete data plus chain reconciliation

Use service unit tests for rules, real MySQL integration tests for SQL/locks, HTTP tests for roles/CSRF and browser tests for visible behavior. Hardware and Testnet need their own real smoke evidence.

Concurrency tests need independent transactions and a start barrier so work genuinely overlaps. Assert final database rows and states, not only one HTTP conflict. Write Given/When/Then, dataset, expected code, result and evidence path for each case. Agree performance targets with the supervisor and distinguish API, tap-wait and chain-confirmation latency.

41 Troubleshooting in a useful order

Symptom	First checks
Boot does not start	JDK/Boot, process environment, datasource and migration logs
Flyway checksum mismatch	Changed historical migration; add a repair migration, do not blindly repair history
401 / 403	Session cookie, identity, roles, CSRF refresh and selected filter chain
API returns HTML	API rewrite order versus SPA fallback
Database/time errors	JDBC URL, initialized schema and UTC convention
Missing shadcn styles	CSS import, Tailwind plugin, alias and CLI configuration
Phone cannot open QR	Public HTTPS address, network and accidental localhost
Java cannot see reader	Driver, Smart Card service, slot and native Windows execution
UID without category	Provisioning/parser/mapping; category is not inherent in UID
Repeated tap events	Removal gate and single-use job consumption
Sui object not found	Network and package/registry/cap deployment consistency
Insufficient gas	Configured address, Testnet balance and faucet
Promise resolved but chain failed	FailedTransaction/result status
Stuck PROCESSING outbox	Expired-lease recovery, worker crash and claim ownership

Trace a browser correlation ID through Backend logs, database state and agent/worker evidence. Change one variable at a time and use mocks to isolate external dependencies before replaying real requests.

Never 'fix' failures by global permitAll, disabling CSRF, removing uniqueness, forcing UI success, replacing every reader error with a test UID or manually labelling an outbox row CONFIRMED.

Acceptance: another person reproduces and recovers three faults using the runbook and can explain the cause.

42 Demonstration and completion criteria

Ten-minute demonstration

1. Explain component locations and visible Mock/Real labels.
2. Admin enrolls a card; a second tap demonstrates duplicate rejection.
3. A phone registers PENJAGA with invalid and valid MRN cases.
4. Staff verifies; a wrong-category card fails; the correct AVAILABLE card succeeds.
5. Show assignment, Issued inventory and Pending proof, followed by confirmation.
6. Verify the digest; demonstrate mismatch only with isolated tampered test data.
7. Return the card and show overdue/retry test evidence.

Completion means a new machine can follow the README, configuration keys are explained, no live secrets are committed, all four registration categories and staff/admin flows operate, and reader recovery and proof reconciliation have evidence. HTTPS, sessions, CSRF, migrations, least privilege and restore procedures must be tested rather than merely configured.

Deliver README, ERD, API contracts, ADRs, test report, UAT checklist, deployment manifest, reader installation instructions, user guide, restore runbook and known limitations. Do not claim hospital integration or physical door access without the corresponding approved interface and test.

Start your implementation with chapters 03-07: directories, toolchain, Initializr, MySQL, ping and health. Then use actual startup logs, pom.xml and observed errors to advance one layer at a time.

A Snippet index and implementation exercises

Chapter	Location	Work still required in the guideline edition
04-07	Root, infra, backend resources	README, secrets injection, profiles and health/security
09	Flyway migrations	Second active constraint, foreign keys and MySQL tests
11-12	Backend auth/config and frontend lib	Authentication provider, handlers, device chain and token lifecycle
13-15	Registration/hospital modules	Conditional fields, mappers, adapters and evidence-bound verification
16-17	Assignment services	Locks, rules, idempotency, audit and rollback tests
18	Analytics/admin modules	Authorization, timezone boundaries, exports and settings
19-23	Frontend	Providers, routes, pages, interaction states and tests
25-28	Reader Agent	Model-specific transport/parser, device auth and packaging
29-32	Audit and Move package	Canonical proof, types, init, table and permission tests
33-34	Worker	Configuration, claims, reconciliation and independent verification
36-37	Deployment files	Build stages, actual domains and cookie/cache checks

These snippets illustrate responsibilities rather than forming a complete application. Commands require the stated working directory, installed dependencies and substituted values. For every helper you implement, explain its input, validation, writes and failure behavior. The separate code edition requested later is a different artifact, not an implicit claim that this guideline contains a complete codebase.

B Official references and version checks (1)

The source handbook checked these official entry points on 20 September 2026. Versions and platform interfaces change; use the dependency locks and compatible documentation selected for your implementation. Business requirements come from your diagrams and project decisions, not these vendor documents.

[R01] Spring Boot system requirements and Initializr.

<https://docs.spring.io/spring-boot/system-requirements.html>

<https://start.spring.io/>

[R02] Vite installation, engines and templates.

<https://vite.dev/guide/>

[R03] Docker Desktop Windows prerequisites.

<https://docs.docker.com/desktop/setup/install/windows-install/>

[R04] WSL command reference.

<https://learn.microsoft.com/en-us/windows/wsl/basic-commands>

[R05] Flyway MySQL support.

<https://documentation.red-gate.com/fd/mysql-277579322.html>

[R06] MySQL generated columns and unique indexes.

<https://dev.mysql.com/doc/refman/8.4/en/create-table-generated-columns.html>

<https://dev.mysql.com/doc/refman/8.4/en/create-index.html>

[R07] Spring Security CSRF and SPA integration.

<https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html>

[R08] shadcn/ui Vite setup.

<https://ui.shadcn.com/docs/installation/vite>

[R09-R10] ACS product and model-specific API manual download entry. A reference manual is not evidence of compatibility with unspecified hardware.

<https://www.acs.com.hk/en/products/342/acr1252u-usb-nfc-reader-iii-nfc-forum-certified-reader/>

[R11] Java 21 PC/SC API.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.smartcardio/javax/smartcardio/package-summary.html>

C Official references and version checks (2)

[R12] Sui CLI installation and platform options.

<https://docs.sui.io/getting-started/onboarding/sui-install>

[R13] Testnet faucet.

<https://faucet.sui.io/>

[R14] Sui build/test/publish onboarding.

<https://docs.sui.io/getting-started/onboarding/hello-world>

[R15] Sui framework APIs; match the installed compiler/framework.

<https://docs.sui.io/references/framework>

[R16] Mysten gRPC client.

<https://sdk.mystenlabs.com/sui/clients/grpc>

[R17] Transaction result handling and confirmation.

<https://sdk.mystenlabs.com/sui/clients/executing>

<https://sdk.mystenlabs.com/sui/transactions/signing-and-execution>

[R18] Typed transaction inputs.

<https://sdk.mystenlabs.com/sui/transactions/reference>

[R19-R20] Railway Spring Boot and MySQL.

<https://docs.railway.com/guides/spring-boot>

<https://docs.railway.com/databases/mysql>

[R21] Vite deployment and SPA routing on Vercel.

<https://vercel.com/docs/frameworks/frontend/vite>

Project inputs: planning/README.md, architecture, requirements/test plan, data-model/API plan, UI specification, Mermaid diagrams and your later red-and-white square-container requirement.

Unresolved integration inputs remain explicit: approved hospital forms/API, reader and card models, category provisioning/mapping, door-controller interface and cloud budget. These do not prevent building mocks and contracts, but they determine whether real integration can pass acceptance.

D WhatsApp updates, return reminders and staff escalation

This is a sensible extension to the flow when the visitor can receive WhatsApp and has chosen that channel. Separate approval from physical handover so every message states only facts already known. At verification, the registration has no final card and the loan clock has not begun. At successful issue, the system knows the assignment/pass reference and the exact due time.

Event sequence

1. Registration: ask for a mobile number in international format and a separate, unchecked choice to receive operational WhatsApp updates. Store when and how the visitor chose it. Provide a way to decline or stop messages. The registration itself still works without WhatsApp.
2. Staff marks VERIFIED: optionally queue an APPROVED template saying the visitor may attend the counter to collect the pass. It must not include a pass number or return deadline yet.
3. Staff scans, assigns and physically hands over the card: commit assignment, due_at and an ISSUE audit event in one database transaction. Then queue PASS_ISSUED containing a non-secret pass/assignment reference and the exact local date, time and time zone for return. Do not send the raw NFC UID, MRN, ward or patient information.
4. Scheduler checks still-open assignments about 30 minutes before due_at; send one REMINDER template. If the loan is shorter than the lead time, use an agreed smaller threshold or rely on the issue message instead of immediately spamming the visitor. Recheck return state before sending.
5. If due_at passes and the assignment remains open, show a prominent overdue alert to staff with the pass reference, time late and authorized contact/action controls. Staff acknowledges and records follow-up. Returning the card closes the alert and stops pending reminders. An optional second escalation to a supervisor must be an explicit policy decision.

The schedule should use saved UTC due_at for comparisons and display the full local calendar date/time, for example '23 Sep 2026, 4:30 PM (MYT, UTC+8)'. A plain 'return before 4:30' can be misunderstood around day boundaries. If loan rules change after issue, do not silently change existing deadlines; an authorized extension needs a recorded new due time and a correction message.

Data and delivery design

Add whatsapp_opt_in, opted_in_at, phone verification status and message preference to the approved registration model. Add notification_jobs with event/assignment ID, template kind, due time, status, attempts and a uniqueness rule for (assignment_id, kind). Add staff_followups with assignment ID, actor, timestamp, disposition and note. Use an outbound worker: the issue database transaction writes the notification job, then the worker calls the approved WhatsApp provider. A provider request accepted for processing is not proof of delivery; track provider status callbacks and show SENT/DELIVERED/FAILED/UNKNOWN accurately.

Keep business actions independent from messaging availability: if WhatsApp fails, the pass remains issued and the staff panel shows a delivery problem and the printed/in-person handover details. Do not repeatedly send the same reminder; retry transient failures with limits. The staff overdue alert is generated from open assignment plus due_at even when WhatsApp was declined, failed or the visitor has no smartphone.

For an FYP, a clearly labelled mock notification adapter and a staff alert are enough to test the flow before obtaining an approved account, template and number. Production business-initiated WhatsApp messages

require recipient permission and approved message templates under the [WhatsApp Business Messaging Policy](<https://whatsappbusiness.com/policy/>) and [Business Platform features](<https://whatsappbusiness.com/products/business-platform-features/>). Utility messages are a plausible category for operational status and time-sensitive reminders; submit actual wording for review under the account's current rules. This is a platform fact; whether the hospital may use this channel for its visitor records is an additional local approval decision.

Example messages

APPROVED: 'Your visitor request [reference] has been approved. Please go to the pass counter to complete verification and collect your pass. No pass has been issued yet.'

PASS_ISSUED: 'Pass [public pass reference] was issued at the counter. Please return it by [date, time and time zone]. If you have already returned it, ignore this message or contact the counter.'

REMINDER: 'Reminder: pass [public pass reference] is due for return at [date, time and time zone]. Please bring it to the pass counter. Contact the counter if you need help.'

Acceptance: no pass number before issue; exactly one intended message per event; no reminder after a recorded return in normal scheduling; an overdue staff alert appears regardless of message status; the visitor's channel choice is respected; no MRN or raw UID appears in templates, logs or provider payloads.